Accessible Onboarding System

Updated System Design: Partner (Vendor) Dashboard, Form Engine and Embeddable Form. Version 2.0, following the Level 1 good practices.

1. What Changed From the Previous Design

The first design described one React app talking to one Express API. The new requirement is that **partners (vendors) choose and configure their own forms**, then place the form on their own website or platform.

Three new parts are added:

New part	Purpose
Partner (Vendor) Dashboard	Where a partner picks a form type, toggles its fields, previews it and copies an embed tag.
Form Engine	Turns a partner's saved settings into a working accessible form. Used by both the dashboard preview and the live embed.
Embeddable Onboarding Form	A small widget that partners add to their site with one tag. It receives settings in and sends events out.

Everything stays JavaScript: React, Vite, Node.js, Express, SQLite, Drizzle ORM and Zod are unchanged.

2. Design Goals (Level 1 Good Practices)

Level 1 practice	What it means in this system
Settings in, events out	The partner configures the form with options (settings in) and is notified when something happens, such as form submitted (events out).
Never break partners	Every release and every published form has a version number. Partners pin a version and updates never surprise them.
No secrets in the browser	Only a public key is used in the browser. Private keys, webhook secrets and database access stay on the server.
Small and accessible	The embed loads fast, works with keyboard and screen readers, and supports English and Amharic.

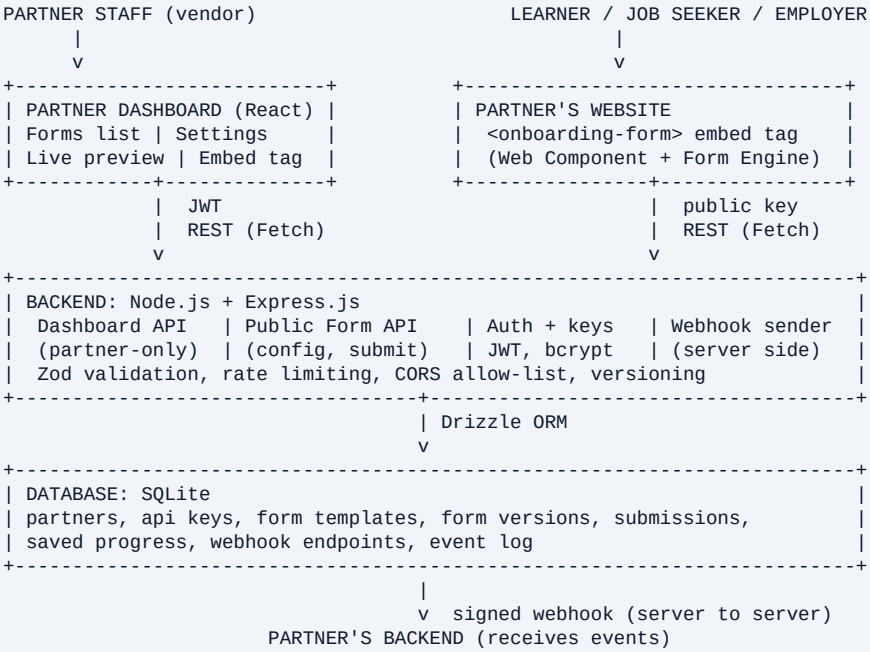
3. Technology Stack (Updated)

Existing layers are unchanged. New or changed layers are marked **(new)**.

Layer	Technology	Purpose
Frontend structure and styling	HTML5, CSS3	Semantic pages, focus indicators, responsive layout.
Language	JavaScript (ES6+)	Frontend and backend.
Dashboard UI	React.js + Vite	Partner dashboard with sidebar, settings panel and preview.
Form Engine (new)	Shared JavaScript package	Reads a form config (JSON) and renders the form. Used by the dashboard preview and the embed.
Embeddable widget (new)	Web Component built with Vite library mode	One-tag embed that works on any partner website without clashing with the partner's CSS.
Backend	Node.js + Express.js	REST API, middleware, partner and public routes.
Database	SQLite + Drizzle ORM	Partners, form configs, versions, submissions and progress.
Validation	Zod	One shared schema validates form configs, dashboard requests and learner submissions.
Authentication (new)	JWT + bcrypt; API keys	Partner staff sign in with JWT. Embed uses a public key. Servers use a secret key.

Layer	Technology	Purpose
Events (new)	Browser CustomEvents + signed webhooks	Browser events for the partner page, webhooks for the partner's server.
Voice and read aloud	Web Speech API	Optional speech input and read-aloud.
Languages	JSON translation files	English and Amharic.
Docs and testing	OpenAPI/Swagger, Vitest, React Testing Library, Node test runner, Supertest, axe-core, Postman	As before, extended to cover the new endpoints and the embed.
Version control	Git + GitHub	As before.

4. System Architecture



The key idea is that the **Form Engine is the same code** in the dashboard preview and in the live embed. What the partner previews is exactly what learners see.

5. The Partner Dashboard (From the Sketch)

5.1 Screen layout

The sketch shows three areas. They map to three React components.

Sketch area	Component	Behaviour
Left sidebar "Forms": Learner, Signup, Job seeker, Employer form	FormsSidebar	Lists the form types. Selecting one loads its settings.
Right panel "Settings": Name, Location, Address, Document ID, Current level, Other +	SettingsPanel	One toggle per field. "Other +" adds a custom field.
Arrow to "Form engine", then the box with "tag ... preview"	PreviewPane + EmbedTagBox	Shows the live form rendered by the Form Engine and the embed tag to copy.

```
+--------------------+-------------------------------------+
| FORMS              | SETTINGS (for selected form)        |
|  > Learner         |  [x] Name                           |
|    Signup          |  [x] Location                       |
|    Job seeker      |  [ ] Address                        |
|    Employer form   |  [x] Document ID                    |
|                    |  [x] Current level                  |
|                    |  [ + Other ]                        |
+--------------------+-------------------------------------+
| EMBED TAG  <onboarding-form ... />      [Copy]           |
| PREVIEW  (rendered by Form Engine, English | Amharic)    |
| +-----------------------------------------------------+  |
| |  live form                                        |  |
| +-----------------------------------------------------+  |
|                      [Save draft]  [Publish]             |
+----------------------------------------------------------+
```

5.2 Partner workflow

1. Partner signs in to the dashboard.
2. Partner selects a form type in the sidebar.
3. Partner turns fields on or off in Settings and may add custom fields with "Other +".
4. The Form Engine redraws the preview instantly.
5. Partner clicks **Save draft** (editable) or **Publish** (creates a new immutable version).
6. Partner copies the embed tag and pastes it into their website.
7. Partner registers a webhook URL to receive events on their server.

5.3 Design decisions for the dashboard

These are proposed decisions for the team to confirm.

- **Locked core fields.** Some fields cannot be switched off for a form type. The toggle is shown as on and disabled with a plain-language reason.
- **Field options per toggle.** Required or optional, label text in English and Amharic, and help text.
- **Draft vs published.** Only published versions are served to learners. Drafts exist only in the dashboard.
- **Dashboard accessibility.** The dashboard itself meets the same keyboard, focus and screen-reader rules as the learner form.

6. Form Engine

The Form Engine takes a **form config** (JSON) and produces an accessible, step-by-step form.

6.1 Form config (settings in)

```
{
  "formId": "frm_8f2a",
  "type": "learner",
  "version": 3,
  "languages": ["en", "am"],
  "defaultLanguage": "en",
  "features": { "voiceInput": true, "readAloud": true },
  "fields": [
    { "key": "name", "enabled": true, "required": true, "locked": true },
    { "key": "location", "enabled": true, "required": false },
    { "key": "address", "enabled": false },
    { "key": "documentId", "enabled": true, "required": true },
    { "key": "currentLevel", "enabled": true, "required": false,
      "options": ["Beginner", "Intermediate", "Advanced"] },
    { "key": "custom_1", "enabled": true, "type": "text",
      "label": { "en": "Preferred contact time", "am": "..." } }
  ],
  "theme": { "accentColor": "#0f766e" }
}
```

6.2 Engine rules

- A field catalog (name, location, address, document ID, current level, and so on) defines each field's input type, validation, labels and help text in both languages.
- The config is validated with Zod on the dashboard, on the server and again in the engine.
- One question or small group per step, review step before submit, plain-language errors.
- Voice input fills a field but never advances the form on its own.
- Save and resume: progress is stored without a timer expiry.
- Custom "Other" fields are limited to safe types (text, choice, yes/no) so partners cannot inject code.

7. Embeddable Form and Event Contract

7.1 Embed tag

```
<script src="https://cdn.example.org/embed/v1/loader.js" async></script>

<onboarding-form
  form-id="frm_8f2a"
  public-key="pk_live_xxxxxxxx"
  form-version="3"
  lang="am">
</onboarding-form>
```

- `v1` in the script path is the **widget version**. `form-version` pins the **form config version**. Both stay stable until the partner chooses to move.
- The widget uses Shadow DOM so partner CSS cannot break it and it cannot break the partner's page.

7.2 Settings in

Setting	Source
form-id, form-version	Embed tag attributes.
lang	Embed tag attribute (en or am).
Fields, labels, features, theme	Fetched as a form config from the Public Form API using the public key.
Optional prefill values	JavaScript property set by the partner (non-secret data only).

7.3 Events out

Browser events are dispatched on the element so the partner page can listen.

Event	When	Payload (no private data)
onboarding:ready	Form loaded and visible.	formId, formVersion
onboarding:step-changed	User moves to another step.	stepIndex, stepKey
onboarding:progress-saved	Progress saved.	submissionId
onboarding:submitted	Final submit succeeded.	submissionId, status
onboarding:error	Load or submit failed.	code, plain-language message

```
document.querySelector('onboarding-form')
  .addEventListener('onboarding:submitted', (e) => {
    console.log('Done', e.detail.submissionId);
  });
```

Server events (webhooks). The same `submitted` event is also sent from the platform's server to the partner's registered webhook URL. The body is signed with HMAC (secret kept on the server), includes a timestamp and an event ID, and is retried on failure. Full learner data is delivered only through webhooks or the secret-key API, never through browser events.

8. Backend Design

8.1 Modules

Module	Responsibility
Auth	Partner staff login (JWT + bcrypt), roles (owner, editor).
Partners	Partner profile, allowed website origins, API keys.
Forms	Form templates, drafts, publishing, version history.
Public Form API	Serves published configs and accepts submissions and progress.
Submissions	Validates with Zod, stores, exposes results to the partner.
Webhooks	Stores endpoints, signs and sends events, retries.
Security middleware	CORS allow-list, rate limiting, security headers, request size limits.

8.2 API endpoints

All routes are versioned under `/api/v1`.

Dashboard API (JWT required)

Method	Path	Purpose
POST	/api/v1/auth/login	Partner staff sign in.
GET	/api/v1/forms	List the partner's forms (Learner, Signup, Job seeker, Employer).
GET	/api/v1/forms/:id	Get a form with its current draft settings.
PUT	/api/v1/forms/:id/draft	Save field settings (toggles, labels, custom fields).
POST	/api/v1/forms/:id/publish	Create a new immutable version.
GET	/api/v1/forms/:id/versions	Version history.
GET, POST	/api/v1/webhooks	List or register webhook endpoints.
POST	/api/v1/keys/rotate	Rotate keys (secret key shown once).
GET	/api/v1/forms/:id/submissions	View submissions for the partner's form.

Public Form API (public key, restricted by allowed origins)

Method	Path	Purpose
GET	/api/v1/public/forms/:formId?version=3	Get the published form config.
POST	/api/v1/public/forms/:formId/progress	Save progress (save and resume).
POST	/api/v1/public/forms/:formId/submit	Submit the completed form.

9. Data Model (Drizzle ORM / SQLite)

Table	Key columns
partners	id, name, status, created_at
partner_users	id, partner_id, email, password_hash, role
api_keys	id, partner_id, public_key, secret_key_hash, status, created_at
allowed_origins	id, partner_id, origin
form_templates	id, type (learner, signup, job_seeker, employer), default_config
partner_forms	id, partner_id, template_id, draft_config, current_version
form_versions	id, partner_form_id, version, config, published_at (immutable)
submissions	id, partner_form_id, form_version, status, data (encrypted fields), created_at

Table	Key columns
onboarding_progress	id, partner_form_id, resume_token_hash, step, partial_data, updated_at
webhook_endpoints	id, partner_id, url, signing_secret, active
event_log	id, partner_id, event_type, submission_id, delivery_status, created_at (no private content)

- `form_versions` rows are never edited after publishing. This is what guarantees partners are not broken.
- Secret keys are stored only as hashes and shown to the partner once.
- Sensitive fields such as Document ID are stored encrypted and are never written to logs.

10. Level 1 Practices Applied (Checklist)

Practice	Implementation	How to verify
Settings in, events out	Form config JSON in, CustomEvents and webhooks out.	Contract tests for each event and each setting.
Never break partners	Widget path /embed/v1/, pinned form-version, immutable version rows, versioned /api/v1 routes. Only additive changes inside a version.	Snapshot test of every published config. Old versions keep rendering after a new release.
No secrets in the browser	Browser uses only the pk_ public key. Secret key, webhook signing secret and DB access are server-only. Public key is limited by allowed origins and rate limits. Sensitive data never placed in browser events.	Code review plus a build check that scans the embed bundle for secret patterns.
Small and accessible	Light bundle, lazy-loaded Speech API code, Shadow DOM, semantic HTML, visible focus, screen-reader labels, English and Amharic.	Bundle size budget in CI, axe-core tests, manual keyboard and screen-reader checks.

11. Security and Privacy

- Treat everything in the browser as public. The public key only allows reading a published config and submitting to that form.
- Check the request origin against the partner's allowed list and apply per-key rate limits.
- Validate every request body with Zod. Reject unknown fields.
- Hash passwords with bcrypt. Keep JWT lifetimes short.
- Sign webhooks with HMAC and include a timestamp to prevent replay.
- Do not log private draft or support content.
- Use HTTPS only. Set security headers.
- Partners see only their own data (every query is scoped by partner_id).

12. Accessibility Requirements

- Keyboard navigation and visible focus everywhere, including the dashboard.
- Labels, help text and error messages are linked to inputs for screen readers.
- Plain-language errors in English and Amharic.
- Voice input is optional and never auto-advances the form. Speech recognition support varies by browser and language, so the typed path must always work.
- Read-aloud is optional.
- Dashboard toggles are real checkboxes or switches with accessible names. Preview updates are announced politely to screen readers.
- Test with axe-core and manually, early and often.

13. Suggested Build Order

1. Shared Zod schemas and the field catalog.
2. Form Engine rendering a config with English and Amharic.
3. Dashboard shell: sidebar, settings panel, live preview.
4. Dashboard API: save draft, publish, versions (Drizzle tables).
5. Public Form API and the embed Web Component.
6. Public key, allowed origins and rate limiting.
7. Events: CustomEvents, then signed webhooks.
8. Accessibility, security and contract tests.
9. API documentation in OpenAPI and a short partner integration guide.

14. Open Questions for the Team

- Which fields are locked (always on) for each of the four form types?
- Can partners add unlimited custom fields, or a fixed maximum?
- Do partners need approval before publishing, or can they publish directly?
- Should the embed also be offered as an iframe for partners who cannot use Web Components?
- Who receives submissions: the partner only, or the platform admin as well?
- Final package versions to be confirmed before installation.